

Daten in bestehende Delta-Tabellen einfügen

Section 2 · Data Ingestion and Loading · Quelle: Kurs 1, Kapitel 16–17

Neben den Methoden zur Erstbefüllung einer Tabelle (CTAS, COPY INTO, Streaming Tables) gibt es einen sehr häufigen Anwendungsfall: neue oder geänderte Datensätze in eine **bereits bestehende** Delta-Tabelle einzupflegen, ohne Duplikate zu erzeugen oder die gesamte Tabelle neu zu schreiben. Dafür bietet Databricks den `MERGE INTO`-Befehl.

1. MERGE INTO: Updates, Inserts und Deletes in einem Schritt

`MERGE INTO` vergleicht eine Quelltable (bzw. das Ergebnis einer Abfrage) mit einer Zieltabelle anhand einer Join-Bedingung und führt je nach Ergebnis unterschiedliche Aktionen aus: Bestehende Zeilen werden aktualisiert (`UPDATE`), gelöscht (`DELETE`) oder neue Zeilen eingefügt (`INSERT`) – alles als eine einzige, atomare Transaktion. Das macht `MERGE INTO` zum Standardwerkzeug für inkrementelle Upsert-Logik, etwa beim Zusammenführen von CDC-Änderungen oder beim regelmäßigen Abgleich einer Bronze- mit einer Silver-Tabelle.

```
MERGE INTO main_users_target target
USING update_users_source source
ON target.id = source.id
WHEN MATCHED AND source.status = 'update' THEN
  UPDATE SET
    target.email = source.email,
    target.status = source.status
WHEN MATCHED AND source.status = 'delete' THEN
  DELETE
WHEN NOT MATCHED THEN
  INSERT (id, first_name, email, sign_up_date, status)
  VALUES (source.id, source.first_name, source.email, source.sign_up_date, source.status);
```

Die Bedingung hinter `WHEN MATCHED` bzw. `WHEN NOT MATCHED` kann zusätzliche Filter enthalten (hier: `source.status`), sodass sich Update-, Delete- und Insert-Fälle in einer einzigen Anweisung fein steuern lassen. Wie jede Delta-Operation wird auch `MERGE INTO` als neue Version in der Tabellenhistorie protokolliert und kann über `DESCRIBE HISTORY` sowie `SELECT * FROM tabelle VERSION AS OF n` (Time Travel) nachvollzogen werden.

2. Schema-Evolution innerhalb von MERGE INTO

Enthält die Quelltable neue Spalten, die in der Zieltabelle noch nicht existieren, schlägt ein normales `MERGE INTO` mit einem Schema-Fehler fehl. Für genau diesen Fall bietet Databricks die Erweiterung `MERGE WITH SCHEMA EVOLUTION INTO`: Sie erlaubt es, neue Spalten aus der Quelle automatisch in die Zieltabelle zu übernehmen, ohne vorher eine Session-Konfiguration setzen zu müssen.

```
-- Ohne Schema-Evolution: schlägt fehl, wenn "new_users_source"
-- eine zusätzliche Spalte "country" enthält, die im Ziel fehlt
MERGE INTO main_users_target target
USING new_users_source source
ON target.id = source.id
WHEN MATCHED AND source.status = 'update' THEN
  UPDATE SET
    target.email = source.email,
    target.status = source.status
WHEN NOT MATCHED AND source.status = 'new' THEN
  INSERT (id, first_name, email, sign_up_date, status, country)
  VALUES (source.id, source.first_name, source.email, source.sign_up_date, source.status,
source.country);

-- Mit Schema-Evolution: die neue Spalte "country" wird automatisch
-- zur Zieltabelle hinzugefügt
MERGE WITH SCHEMA EVOLUTION INTO main_users_target target
USING new_users_source source
ON target.id = source.id
WHEN MATCHED AND source.status = 'update' THEN
  UPDATE SET
    target.email = source.email,
    target.status = source.status
WHEN NOT MATCHED AND source.status = 'new' THEN
  INSERT (id, first_name, email, sign_up_date, status, country)
  VALUES (source.id, source.first_name, source.email, source.sign_up_date, source.status,
source.country);
```

3. Weitere Ingestion-nahe Plattform-Features

Über die in diesem Themenordner behandelten Ingestion-Methoden hinaus bietet Databricks weitere Funktionen, die bei wachsender Architektur relevant werden können:

- **Lakehouse Federation** – ermöglicht Abfragen auf externe Datenquellen (z. B. andere Datenbanken) direkt aus Databricks heraus, ohne die Daten vorher physisch zu kopieren.

- **Delta Sharing & Databricks Marketplace** – ein offener Standard bzw. ein Marktplatz, um Datensätze sicher mit anderen Organisationen zu teilen oder von dort zu beziehen, ebenfalls ohne unnötige Datenkopien.
- **Zerobus** – eine Möglichkeit, Daten sehr niedrig-latent direkt in Databricks-Tabellen zu streamen (z. B. aus IoT- oder Event-Quellen).

Ergänzt aus der offiziellen Databricks-Dokumentation: Für `MERGE WITH SCHEMA EVOLUTION INTO` empfiehlt Databricks inzwischen ausdrücklich, die Schema-Evolution direkt in der jeweiligen Schreiboperation anzugeben (wie oben gezeigt), statt sie global über eine Session-Konfiguration zu aktivieren – das macht das Verhalten pro Anweisung nachvollziehbar und vermeidet unbeabsichtigte Schema-Änderungen an anderer Stelle. Die Funktion steht ab Databricks Runtime 15.2 zur Verfügung; Nutzerinnen und Nutzer, die ausschließlich LTS-Versionen einsetzen, benötigen Runtime 15.4 LTS als erste LTS-Version mit dieser Funktion.

Quelle: [Update table schemas with schema evolution – Databricks-Dokumentation](#)